# Failure (bad key + cost comparison) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint builds toward the full solution.


- Hint 1 — The two parts, and the exact pieces
- Hint 2 — The plan, and almost the whole thing


## Hint 1 — The two parts, and the exact pieces

Part 1 tests that you catch `openai.AuthenticationError` *specifically*, not just any exception — because a bare `except Exception` would also hide a real bug in your own code behind the same generic message, making it much harder to tell "bad key" apart from "I broke something." `OpenAI(api_key="sk-fake-key-123")` — passing a bad key directly to the client constructor (instead of relying on the real one from `.env`) forces an authentication failure on the very next call.

Part 2 uses `response.usage` — every response object carries back the exact token counts OpenAI billed you for, split into `prompt_tokens`, `completion_tokens`, and `total_tokens`. Comparing this number between a 1-sentence and a 20-sentence system prompt turns "shorter prompts cost less" from a rule you were told into a number you measured yourself.

## Hint 2 — The plan, and almost the whole thing

```
function try_bad_key():
    bad_client = OpenAI(api_key="sk-fake-key-123")
    try: send a message
    except openai.AuthenticationError: print a clean message

function compare_prompt_cost(question):
    short = call with a short system prompt
    long = call with a long system prompt
    print short.usage.total_tokens
    print long.usage.total_tokens
```

```
# auth_and_cost_practice.py
import openai
```

```
from openai import OpenAI

client = OpenAI()

def try_bad_key() -> None:
    bad_client = OpenAI(api_key="sk-fake-key-123")
    try:
        bad_client.chat.completions.create(
            model="gpt-4o-mini",
            messages=[{"role": "user", "content": "hi"}],
        )
    except openai.AuthenticationError:
        print("Login failed — check your API key.")

def compare_prompt_cost(question: str) -> None:
    short_prompt = "Be helpful."
    long_prompt = (
        "You are a friendly, detailed, thorough assistant "
        "who explains everything carefully. "
    ) * 10

    short_messages = [
        {"role": "system", "content": short_prompt},
        {"role": "user", "content": question},
    ]
    long_messages = [
        {"role": "system", "content": long_prompt},
        {"role": "user", "content": question},
    ]
    short = client.chat.completions.create(
        model="gpt-4o-mini", messages=short_messages
    )
    long = client.chat.completions.create(
        model="gpt-4o-mini", messages=long_messages
    )

    print(f"Short system prompt: {short.usage.total_tokens} tokens")
    print(f"Long system prompt:  {long.usage.total_tokens} tokens")
```

Call both functions and confirm the long prompt uses noticeably more tokens, then compare against the Solution.

Full solution: Show me the solution